■■ React for Kids

Intro to Components

■ **Subject**	Web Development with React
■ **Level**	Beginner – Kids / Teens
■■ **Duration**	45 – 60 minutes
■ **Context**	India-themed analogies (tiffin dabba, mithai, train)
■ **Topics**	Components, JSX, App Component, Reusability

■ Learning Objectives

- ■ Understand what React is and why it was created
- ■ Define a component and explain it using real-life Indian analogies
- ■ Identify the three parts of a React component (function, return, JSX)
- ■ Write a simple functional component with a return statement
- ■ Reuse components inside an App component
- ■ Appreciate how reusability saves time in coding

■ Table of Contents

1 What is React?

React is a free, open-source JavaScript **library** created by engineers at Facebook in 2013. It helps developers build fast, interactive web applications by breaking the user interface into small, manageable pieces called **components**. Today, React powers apps like Instagram, Netflix, and WhatsApp Web.

■ **Lego Analogy**	*Think of React like a box of Lego bricks. Each brick is a component. You snap them together to build something amazing — and you can reuse the same bricks in different projects!*

Key Facts

■ **Created by**	Facebook (Meta) engineers – released publicly in 2013
■ **Type**	JavaScript library (not a full framework)
■ **Specialty**	Building reusable UI components
■ **Used by**	Instagram, Netflix, Airbnb, WhatsApp Web, and thousands more

2 What is a Component?

A **component** is a small, self-contained piece of a webpage. Each component handles one job — showing a heading, a button, an image, or a product card. When you put many components together, you get a complete web application.

■ **Tiffin Dabba**	Each compartment of a tiffin box holds one food — dal, rice, sabzi, roti. Each React component holds one piece of the page.
■ **Train Compartments**	An Indian train has compartments for sleeping, sitting, and luggage. Each compartment has its own job, yet together they form one train.
■ **Rubber Stamp**	A rubber stamp lets you print the same design over and over. Components let you write code once and reuse it everywhere.

3 Anatomy of a React Component

Every functional React component has three essential parts. Understanding each part will help you read and write components confidently.

① Function Declaration

Defines the component as a JavaScript function. The name MUST start with a Capital Letter — just like your name in the class attendance register!

```
function Greeting() { // ← Capital G required
```

② Return Statement

The return keyword sends back the HTML-like content that will appear on the screen. Think of the sweet-wala handing you a gulab jamun — the return statement hands the page its content.

```
return Namaste!;
```

③ JSX (JavaScript XML)

JSX looks like HTML but lives inside JavaScript. React converts it into real DOM elements. You can write tags like , , inside JS!

```
// Namaste! ← This IS JSX
```

4 Code Example: Greeting Component

Let's look at our very first complete React component. Read each line carefully and notice how the three parts we just learned appear in the real code.

```
// A simple React component

function Greeting() {

return (

<h1>Namaste! Welcome to React!</h1>

);

}
```

`function Greeting()`	Function declaration — capital G tells React it's a component
`return (...)`	Return statement — everything inside the () appears on screen
`...`	JSX — HTML-like tag that React converts to a real heading

5 Reusing Components — The Rubber Stamp Magic

One of React's superpowers is **reusability**. Once you write a component, you can use it as many times as you want — just like stamping the same ink design on multiple pages without redrawing it every time.

```
// Using Greeting three times — no copy-paste needed!

function App() {

return (

<div>

<Greeting /> {/* stamp 1 */}

<Greeting /> {/* stamp 2 */}

<Greeting /> {/* stamp 3 */}

</div>

);
```

```
    }
```

■ Why Reusability Matters	*Imagine you have 50 product cards on a shopping website. Without components you'd write the same HTML 50 times. With React, write it ONCE — use it 50 times. Change one component → every instance updates automatically!*

6 The App Component — Your Mithai Box

Every React project has a special component called **App**. It is the top-level container — the *mithai box* — that holds all the other components. When React starts your app, it looks for App and renders it first.

App (Mithai Box ■)

Header | Main Content | Footer

Logo | NavBar | Article | Sidebar | SocialLinks

7 Code Example: App Using Greeting Components

```
// 1. Define the Greeting component
function Greeting() {
return <p>■ Namaste! Welcome to React!</p>;
}

// 2. Define the App component (the mithai box)
function App() {
return (
<div>
<h1>My First React App</h1>
<Greeting /> {/* use Greeting here */}
<Greeting /> {/* use it again! */}
</div>
);
}
```

Output on screen: My First React App | ■ Namaste! Welcome to React! ×2

8 Guided Practice

■ Fill in the Blanks

Complete the code below to write your first React component. Remember: the function name must start with a Capital Letter!

```
function _______() {

return <h2>Namaste!</h2>;

}
```

Q1. What should go in the blank? (Hint: it should start with a capital letter)

Answer: ________________________________

Q2. What keyword sends content back to the screen?

Answer: ________________________________

Q3. What do we call the HTML-like code inside JavaScript?

Answer: ________________________________

Q4. If you wanted to show this greeting twice, how would you write it in App?

Answer: ________________________________

9 Mini-Challenge: Build Your Namaste Card

Now it's your turn to be a React developer! Follow the two steps below to build your very own Namaste Card app.

■ Step 1 – Create a NameCard Component

Write a NameCard component that displays your name inside a <p> tag. Use function NameCard() and return your name.

```
function NameCard() {

return <p>My name is _______ ■</p>;

}
```

■ Step 2 – Combine Components in App

Put both the Greeting component AND your new NameCard component inside the App component. They should both appear on screen.

```
function App() {

return (

<div>

<Greeting />

<NameCard />

</div>

);

}
```

■ Bonus Challenge

Can you add a third component called FavSubject that shows your favourite school subject? Place it inside App after NameCard!

10 Recap & Key Terms Glossary

■■ **React**	A JavaScript library for building fast, component-based UIs
■ **Component**	A small, reusable piece of a webpage (function + return + JSX)
■ **JSX**	HTML-like syntax used inside JavaScript in React
■ **Reusability**	Write a component once; use it as many times as needed
■ **App**	The root component that holds all other components inside it
■■ **Capital Name**	React component function names MUST start with a capital letter
■■ **Return**	The keyword that sends content to be displayed on the page

11 Bonus: Fun React Facts! ■

■ Born in 2013

React was created by Jordan Walke at Facebook and open-sourced in May 2013 at JSConf US.

■ Instagram

Instagram's entire web app was rewritten in React — one of the earliest big adoptions.

■ WhatsApp Web

The web version of WhatsApp uses React for its fast, real-time message updates.

■ Netflix

Netflix uses React on its TV UI — fast rendering is critical for a smooth watching experience.

■ Virtual DOM

React uses a 'Virtual DOM' — an invisible copy of the page — to figure out the smallest possible update needed, making it super fast.

■ Popularity

React is consistently ranked as the most-used front-end library in Stack Overflow's annual Developer Survey.

12 Exit Ticket

Before you leave today's lesson, answer these questions in your own words. There are no wrong answers — just think carefully!

■ **Easy**	What is a React component? Use one of today's analogies in your answer.
	Your Answer: ____
	____
■ **Medium**	What are the three parts of a React component? Give a one-line description of each.
	Your Answer: ____
	____
■ **Hard**	Why is reusability important in programming? Give a real-life example.
	Your Answer: ____
	____
■ **Star**	Can a component go inside another component? How did we do this today?
	Your Answer: ____
	____

■ Great work, young coder! Keep exploring React!